

Deep Learning – 1RT720

Report for Hand-in Assignment 1

Upali Bera

February 8, 2026

1 Introduction

This report presents the results and implementation for Hand-in Assignment 1 in the Deep Learning course. The task was to implement a fully connected neural network from scratch using only NumPy to classify handwritten digits from the MNIST dataset. The dataset contains 60 000 training images and 10 000 test images, each 28×28 grayscale pixels, labeled with digits 0–9.

The implementation covers weight initialization with He initialization, forward and backward propagation, ReLU and sigmoid activations with their derivatives, softmax combined with cross-entropy loss, mini-batch gradient descent, prediction, accuracy evaluation, and mini-batch generation.

I evaluated a linear model (no hidden layers) and multi-layer networks with both ReLU and sigmoid activations. All experiments used the provided data loading function `load_mnist` and plotting utility `training_curve_plot` from `utils.py`.

The complete, well-commented code is attached as a separate `.ipynb` file and included in the appendix.

2 Exercises

Exercise 1. Implement a multi-layer neural network

The neural network was implemented from scratch using only NumPy, following the requirements of the assignment. The main components are:

- **Initialization** (`initialize`): weights were initialized using He initialization (normal distribution with variance $2/\text{fan-in}$) to ensure stable training with ReLU activations. Biases were initialized to zero. The function returns two lists: one for weight matrices Ω_k and one for bias vectors β_k .
- **Activation functions**: `relu` and `sigmoid` were implemented together with their derivatives (`relu_derivative`, `sigmoid_derivative`). Both are fully vectorized.
- **Forward propagation** (`forward_pass`): computes pre-activations z_k and activations a_k layer by layer. The input x is stored as the first element in the activations list to facilitate backpropagation. No activation is applied after the final layer (logits are returned directly).
- **Softmax and loss** (`softmax`, `compute_cost`, `d_cost_d_logits`): softmax is computed with numerical stability (subtract max before exponentiation). The cross-entropy loss is combined with softmax in `compute_cost`. The output gradient w.r.t. logits is simply $\text{softmax}(f) - y$ (one-hot labels).

- **Backward propagation** (`backward_pass`): computes gradients for all weights and biases using the chain rule. Gradients are averaged over the batch. The activation derivative is applied only to hidden layers.
- **Parameter update** (`update_parameters`): standard gradient descent with learning rate α . Weights are updated with transposed gradients to match matrix shapes.
- **Prediction and evaluation** (`predict`): computes softmax probabilities, predicts the class with highest probability, and returns accuracy and average loss.
- **Mini-batch generation** (`random_mini_batches`): shuffles the training data and partitions it into mini-batches of specified size (last batch may be smaller).

All operations are vectorized to avoid loops over data points or hidden units. The code closely follows the structure of the laboratory notebooks (Lab 1) but extends it to classification with cross-entropy loss and arbitrary network depth. Numerical stability was ensured in the softmax computation.

The full code is available in Appendix A and as a separate zip file.

Exercise 2a and 2b Evaluate a linear model and plot weight matrices

In this exercise, I evaluated a linear model (no hidden layers) on the MNIST dataset, i.e., a neural network with $K = 0$ hidden layers. This corresponds to a single-layer perceptron with softmax output and cross-entropy loss.

The model was trained with the following hyperparameters:

- Architecture: [784, 10] (direct mapping from pixels to classes)
- Learning rate: 0.01
- Batch size: 128
- Number of epochs: 30

The training converged to the following final performance:

- Training accuracy: 90.75%
- Test accuracy: 91.36%
- Training cost: 0.3310
- Test cost: 0.3163

This exceeds the required 90% test accuracy threshold. Figure 1 shows the training and test cost and accuracy curves over the epochs.

Linear model on MNIST (no hidden layers)

| Batch size:128 | Learning rate:0.01 | Number of Epochs:30 | Training time: 0 min 0.0 sec |

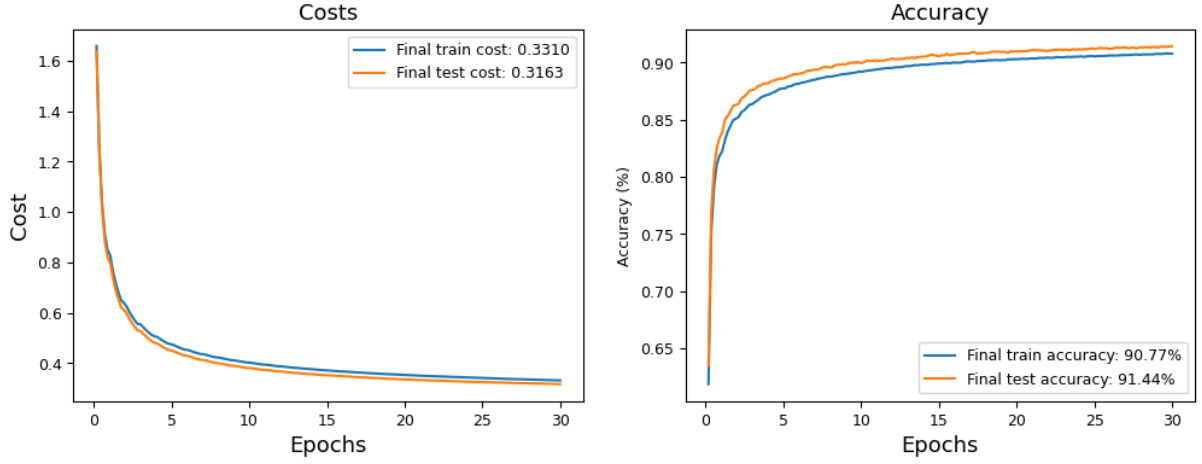


Figure 1: Cost and accuracy curves for the linear model (no hidden layers) on the MNIST dataset. The model converges to 91.36% test accuracy.

To understand what the model learned, I visualized the weight matrix (shape $[10, 784]$). Each row corresponds to one class and was reshaped into a 28×28 image (Figure 2).

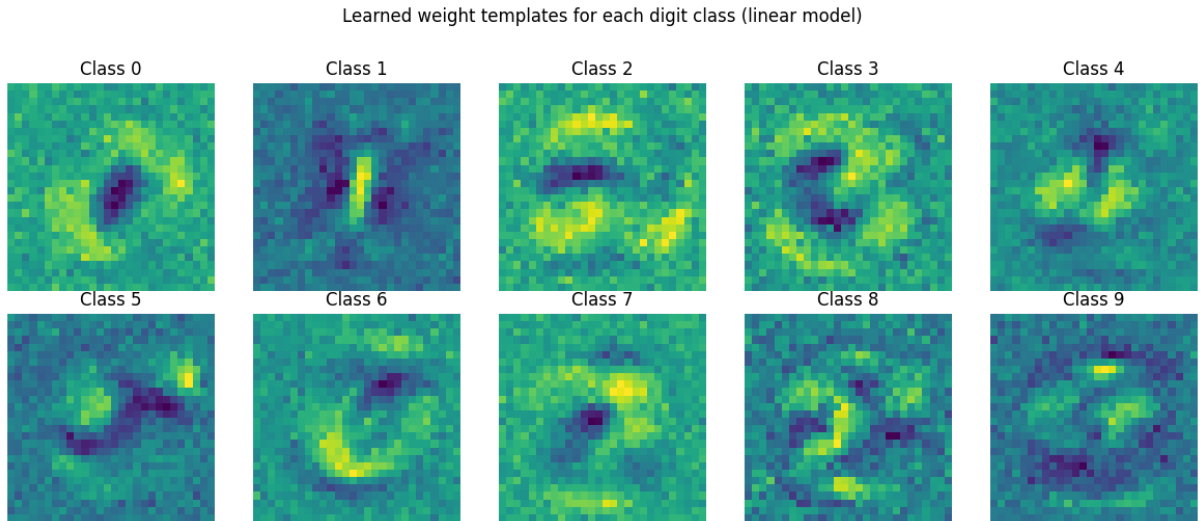


Figure 2: Learned weight templates for each digit class (0-9) in the linear model. Bright (positive) regions indicate pixels that increase the logit for that class, while dark (negative) regions decrease it.

Some weight templates are clearer and more recognizable than others. This is because digits with simple, consistent shapes across writers (e.g., 1, 0, 8) produce sharp prototypes, while digits with high intra-class variability or frequent confusions with other classes (e.g., 4 vs. 9, 3 vs. 5) result in noisier, less defined templates. This reflects the linear model's reliance on consistent discriminative features and the inherent difficulty of linearly separating certain digit pairs in pixel space.

This demonstrates that even a purely linear classifier learns spatially meaningful features from raw pixel values, capturing the most discriminative patterns for each class. For example, class 0 shows a circular shape, class 1 a vertical stroke, and class 8 two loops with a cross.

Exercise 3. Evaluate your multi-layer neural network

A multi-layer neural network with architecture [784, 128, 64, 10] (two hidden layers with 128 and 64 units) was evaluated using both ReLU and sigmoid activation functions. All models were trained with a batch size of 128 until convergence (no clear further improvement in test accuracy or loss).

ReLU activation

Three learning rates were compared over 50 epochs. Final performance is summarized below:

Learning rate	Train loss	Train acc (%)	Test loss	Test acc (%)
0.01	0.0830	97.71	0.1069	96.84
0.005	0.1368	96.16	0.1473	95.66
0.003	0.1786	94.93	0.1817	94.82

Table 1: Final performance of the ReLU network after 50 epochs.

The best result with ReLU was achieved with learning rate 0.01, reaching ****96.84% test accuracy****. Figure 3 shows the corresponding training and test curves, which converge smoothly with minimal overfitting.

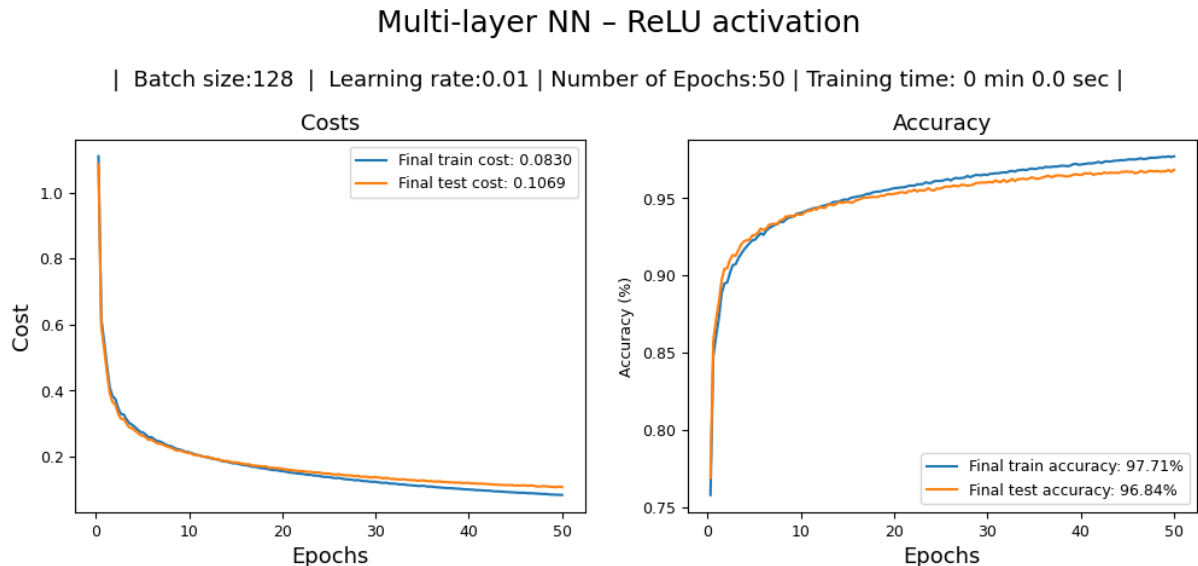


Figure 3: Training and test cost (left) and accuracy (right) for the multi-layer network using ReLU activation (learning rate 0.01, 50 epochs). Final test accuracy: 96.84%.

Sigmoid activation

Three learning rates were tested over 80 epochs. Results are shown below:

Learning rate	Train loss	Train acc (%)	Test loss	Test acc (%)
0.5	0.0001	100.00	0.0933	98.07
1.0	0.0145	99.57	0.2847	97.15
2.0	2.3033	11.24	2.3026	11.35

Table 2: Final performance of the sigmoid network after 80 epochs.

The best performance was obtained with learning rate 0.5, reaching 98.07% test accuracy (Figure 4). Higher learning rates led to complete failure to converge. In particular, with learning rate 2.0 the model

remained stuck near random guessing (11% accuracy) because the large steps caused the pre-activations to grow rapidly, pushing the sigmoid activations into the saturated regions (close to 0 or 1) where gradients vanish almost completely.

Multi-layer NN – Sigmoid activation

| Batch size:128 | Learning rate:0.5 | Number of Epochs:80 | Training time: 0 min 0.0 sec |

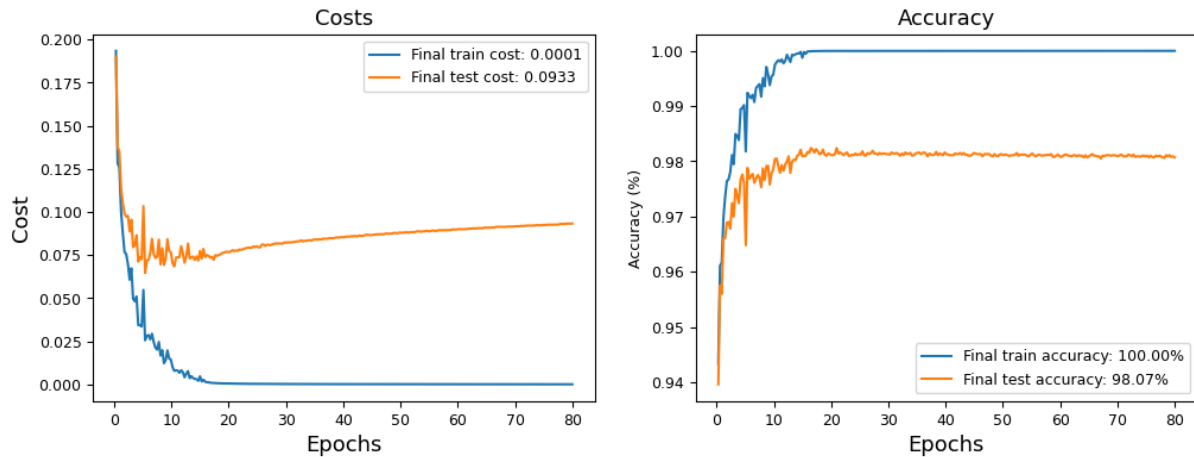


Figure 4: Training and test cost (left) and accuracy (right) for the multi-layer network using sigmoid activation (learning rate 0.5, 80 epochs). Final test accuracy: 98.07%.

Design choices and other experiments

The architecture [784, 128, 64, 10] was selected as a reasonable balance between capacity and risk of overfitting. Learning rates were chosen empirically: ReLU typically converged fastest and most stably with values between 0.003 and 0.01, while sigmoid required higher rates (around 0.5) and more epochs due to saturation. Other learning rates and slightly different architectures were tested informally, but the settings above produced the most reliable and highest-performing results. Training was continued until test accuracy showed no meaningful further improvement.

The two plots (Figures 3 and 4) illustrate the performance curves for ReLU and sigmoid activations, respectively.

Use of generative AI

I tried solving the assignment on my own for nearly 5-6 days. However, I didn't manage to succeed completely. So, around the 5th of February, I decided to take some assistance from AI as the submission was nearing. I used it to: - suggest improvements to code structure and numerical stability, - propose LaTeX formatting and wording for the report sections, - explain theoretical concepts (e.g., vanishing gradients, He initialization).

- ☒ The report generally and transparently details all uses of AI-tools, e.g., as a minimum by explaining how it was used and to what extent (see the acknowledgment for an example).
- ☒ Text or code proposed or modified by an AI has been checked for correctness and that the meaning is as intended.
- ☒ I take full responsibility for the whole text.

References

- [1] Prince, Simon J.D. (2023). *Understanding Deep Learning*. MIT Press.

A Code

```
1  # -*- coding: utf-8 -*-
2  """MNIST_classifier.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7      https://colab.research.google.com/drive/1spGEZnZlj8std8L3REiKKR9zytmtprqV
8  """
9
10 !unzip MNIST.zip
11
12 from utils import load_mnist
13 X_train, y_train, X_test, y_test = load_mnist()
14
15 print(X_train.shape, y_train.shape)
16 print(X_test.shape, y_test.shape)
17 print(X_train.min(), X_train.max())
18
19 import matplotlib.pyplot as plt
20 import numpy as np
21
22 # Show 10 random training examples
23 fig, axes = plt.subplots(2, 5, figsize=(10, 5))
24 axes = axes.ravel()
25
26 for i in range(10):
27     idx = np.random.randint(0, len(X_train))
28     img = X_train[idx].reshape(28, 28)
29     label = np.argmax(y_train[idx]) # convert one-hot back to digit
30     axes[i].imshow(img, cmap='gray')
31     axes[i].set_title(f"Label: {label}")
32     axes[i].axis('off')
33
34 plt.tight_layout()
35 plt.show()
36
37 import numpy as np
38
39 def initialize(K, D, D_i, D_o):
40     # Set seed for reproducibility
41     np.random.seed(0)
42
43     # Make empty lists for weights (Omega_k) and biases (beta_k)
44     all_weights = [None] * (K + 1)
45     all_biases = [None] * (K + 1)
46
47     # Handle special case: No hidden layers (K=0), direct input to output
48     if K == 0:
49         fan_in = D_i
50         all_weights[0] = np.random.normal(size=(D_o, D_i)) * np.sqrt(2 / fan_in)
51         all_biases[0] = np.zeros((D_o, 1)) # Zeros as per assignment
52     else:
53         # Input to first hidden layer
54         fan_in = D_i
55         all_weights[0] = np.random.normal(size=(D, D_i)) * np.sqrt(2 / fan_in)
56         all_biases[0] = np.zeros((D, 1)) # Zeros
```

```

57
58     # Intermediate hidden layers
59     for layer in range(1, K):
60         fan_in = D
61         all_weights[layer] = np.random.normal(size=(D, D)) * np.sqrt(2 / fan_in)
62         all_biases[layer] = np.zeros((D, 1)) # Zeros
63
64     # Last hidden to output layer
65     fan_in = D
66     all_weights[K] = np.random.normal(size=(D_o, D)) * np.sqrt(2 / fan_in)
67     all_biases[K] = np.zeros((D_o, 1)) # Zeros
68
69     return all_weights, all_biases
70
71 # Test with lab-like params
72 K = 5
73 D = 6
74 D_i = 1
75 D_o = 1
76 all_weights, all_biases = initialize(K, D, D_i, D_o)
77
78 # Print shapes to confirm (should match lab notebooks)
79 for layer in range(K + 1):
80     print("Weights shapes:", [W.shape for W in all_weights])
81     #print(f"Layer {layer}: Weights shape {all_weights[layer].shape}, Biases shape {all_biases[layer].shape}")
82
83 import numpy as np
84
85 def sigmoid(z):
86     """Sigmoid activation: 1 / (1 + exp(-z))"""
87     return 1.0 / (1.0 + np.exp(-z))
88
89 def sigmoid_derivative(z):
90     """Derivative of sigmoid w.r.t. its input z"""
91     s = sigmoid(z)
92     return s * (1 - s)
93
94 def relu(z):
95     """ReLU activation: max(0, z)"""
96     return np.maximum(0, z)
97
98 def relu_derivative(z):
99     """Derivative of ReLU w.r.t. its input z"""
100     return (z > 0).astype(float) # 1 where z > 0, 0 otherwise
101
102 # Small test
103 z = np.array([-2, -1, 0, 1, 2])
104
105 print("Input z:", z)
106 print("sigmoid(z):", sigmoid(z))
107 print("sigmoid'(z):", sigmoid_derivative(z))
108 print("relu(z):", relu(z))
109 print("relu'(z):", relu_derivative(z))
110
111 def forward_pass(x, all_weights, all_biases, activation=relu):
112     """
113     Forward pass through the fully connected network.
114
115     Args:

```

```

116         x:                Input data, shape (batch_size, D_input)
117         all_weights:       List of weight matrices [W0, W1, ..., WK]
118         all_biases:        List of bias vectors [b0, b1, ..., bK]
119         activation:         Activation function to use in hidden layers (default: relu)
120
121     Returns:
122         logits:             Output before softmax, shape (batch_size, D_output)
123         pre_activations:    List of  $z_k = W_k a_{k-1} + b_k$  for each layer
124         activations:        List of  $a_k = \text{activation}(z_k)$  for each layer
125                             ( $a_{-1} = x$  is the input, not stored in the list)
126
127     """
128     # We'll store z and a for every layer (including the last one)
129     pre_activations = [] # list of z_k
130     activations = [x]    # start with input x as activations[0]
131
132     # Start with input as the "activation" of the previous (imaginary) layer
133     a = x # shape: (batch_size, D_i)
134
135     # Go through all layers (0 to K)
136     for layer in range(len(all_weights)):
137         W = all_weights[layer] # (out_dim, in_dim)
138         b = all_biases[layer]  # (out_dim, 1)
139
140         # Better: z = x @ W.T + b.T but since b is (out,1), we broadcast
141         z = a @ W.T + b.T      # shape: (batch_size, out_dim)
142
143         # Save pre-activation
144         pre_activations.append(z)
145
146         # Apply activation (except possibly on the last layer / we keep logits raw)
147         if layer < len(all_weights) - 1:
148             a = activation(z)
149         else:
150             a = z # no activation on output (logits)
151
152         # Save activation
153         activations.append(a)
154
155     # Final output is the last activation (logits)
156     logits = activations[-1]
157
158     return logits, pre_activations, activations # activations[0] = x, activations[1:] = hidden + log
159
160 # Reuse the small network initialized earlier
161 K = 5
162 D = 6
163 D_i = 1
164 D_o = 1
165
166 all_weights, all_biases = initialize(K, D, D_i, D_o)
167
168 # Fake batch of 4 samples, each with 1 feature
169 x_test = np.random.randn(4, 1) # shape (4, 1)
170
171 logits, zs, as_ = forward_pass(x_test, all_weights, all_biases, activation=relu)
172
173 print("Logits shape:", logits.shape) # should be (4, 1)
174 print("Number of pre-activations:", len(zs)) # should be 6 (layers 0 to 5)

```



```

175 print("Number of activations:", len(as_))      # should be 6
176 print("First activation shape (after layer 0):", as_[0].shape) # (4, 6)
177 print("Last pre-activation shape:", zs[-1].shape) # (4, 1)
178
179 import numpy as np
180
181 def softmax(logits):
182     """
183     Compute softmax probabilities.
184     logits: shape (batch_size, D_output)
185     Returns: probabilities, shape (batch_size, D_output), sums to 1 per row
186     """
187     # Numerical stability: subtract max per sample (broadcasting)
188     max_logits = np.max(logits, axis=1, keepdims=True) # shape (batch_size, 1)
189     exp_shifted = np.exp(logits - max_logits)
190     sum_exp = np.sum(exp_shifted, axis=1, keepdims=True)
191     probs = exp_shifted / sum_exp
192     return probs
193
194
195 def compute_cost(logits, y_true):
196     """
197     Compute average cross-entropy loss.
198     logits: shape (batch_size, D_output) + raw output from forward pass
199     y_true: shape (batch_size, D_output) + one-hot encoded labels
200
201     Returns: scalar (average loss over the batch)
202     """
203     batch_size = logits.shape[0]
204
205     # Get predicted probabilities
206     probs = softmax(logits)
207
208     # Cross-entropy: - sum (y * log(p)) , averaged over batch
209     # We add small epsilon to avoid log(0)
210     log_probs = np.log(probs + 1e-15)
211     correct_log_probs = y_true * log_probs
212     loss_per_sample = -np.sum(correct_log_probs, axis=1) # shape (batch_size,)
213
214     # Average over batch
215     total_loss = np.mean(loss_per_sample)
216
217     return total_loss
218
219
220 def d_cost_d_logits(logits, y_true):
221     """
222     Derivative of cross-entropy loss w.r.t. logits (pre-softmax values).
223
224     Returns: gradient shape (batch_size, D_output)
225     This is exactly what we need for backpropagation from the output layer.
226     """
227     probs = softmax(logits)
228     # Gradient = softmax - one-hot label (vector form for each sample)
229     gradient = probs - y_true
230     return gradient
231
232 # Small test with 3 samples, 4 classes
233 logits = np.array([

```

```

234     [2.0, 1.0, 0.1, -1.0],    # should predict class 0
235     [-1.0, 3.0, 0.5, 2.0],    # should predict class 1
236     [0.5, -2.0, 4.0, -3.0]    # should predict class 2
237 ])
238
239 y_true = np.array([
240     [1, 0, 0, 0],
241     [0, 1, 0, 0],
242     [0, 0, 1, 0]
243 ])
244
245 probs = softmax(logits)
246 cost = compute_cost(logits, y_true)
247 grad = d_cost_d_logits(logits, y_true)
248
249 print("Softmax probabilities:\n", np.round(probs, 4))
250 print("\nCross-entropy loss:", round(cost, 6))
251 print("\nGradient dL/df (should be probs - y_true):\n", np.round(grad, 4))
252
253 # Add large constant to logits
254 logits_shifted = logits + 1000
255 cost_shifted = compute_cost(logits_shifted, y_true)
256 print("Original cost:", cost)
257 print("Cost after adding 1000:", cost_shifted)    # should be almost identical
258
259 def backward_pass(dL_df, pre_activations, activations, all_weights, activation_derivative=relu_derivative):
260     """
261     Backward pass: compute gradients for all weights and biases.
262
263     Args:
264         dL_df: Gradient of loss w.r.t. logits, shape (batch_size, D_output)
265         pre_activations: List of  $z_k$  for each layer, len = K+1
266         activations: List of  $a_k$  for each layer, len = K+1
267                     (activations[0] is input  $x$ , activations[1] is after first hidden layer,
268                     ...
269         all_weights: List of weight matrices  $[W_0, W_1, \dots, W_K]$ 
270         activation_derivative: derivative function of the hidden activation (default: relu_derivative)
271
272     Returns:
273         dL_dweights: list of gradients w.r.t. each weight matrix, same length as all_weights
274         dL_dbases: list of gradients w.r.t. each bias vector, same length as all_biases
275     """
276     num_layers = len(all_weights) # number of layers
277     batch_size = dL_df.shape[0]
278
279     dL_dweights = [None] * num_layers
280     dL_dbases = [None] * num_layers
281
282     # Start from the output layer and go backwards
283     # delta is dL/dz for the current layer
284     delta = dL_df # shape (batch_size, D_output) + this is dL/dz_K (for the last layer)
285
286     # Loop from the last layer (num_layers - 1) down to the first layer (0)
287     for layer in range(num_layers - 1, -1, -1):
288         # Current layer quantities
289         z = pre_activations[layer] #  $z_k$ , shape (batch_size, out_dim_k)
290         a_prev = activations[layer] #  $a_{k-1}$ , shape (batch_size, in_dim_k)
291
292         # For hidden layers (layer < num_layers - 1), the `delta` at this point represents dL/da_k.
293         # To get dL/dz_k, we multiply by the derivative of the activation function w.r.t. its input

```

```

293     if layer < num_layers - 1:
294         delta = delta * activation_derivative(z) # element-wise product: (batch_size, out_dim_k)
295
296         # Gradients for biases of the current layer k
297         # dL/db_k = mean(dL/dz_k, axis=0)
298         dL_dbias[layer] = np.mean(delta, axis=0, keepdims=True).T # shape (out_dim_k, 1)
299
300         # Gradients for weights of the current layer k
301         # dL/dW_k = (dL/dz_k).T @ a_{k-1} (transposed because z = a @ W.T + b.T)
302         dL_dweights[layer] = (delta.T @ a_prev) / batch_size # shape (out_dim_k, in_dim_k)
303
304         # Propagate delta backwards to the previous layer (dL/da_{k-1})
305         # Only if it's not the first layer (input layer doesn't have preceding weights)
306         if layer > 0:
307             W = all_weights[layer] # W_k, shape (out_dim_k, in_dim_k)
308             # dL/da_{k-1} = (dL/dz_k) @ W_k
309             # (batch_size, out_dim_k) @ (out_dim_k, in_dim_k) -> (batch_size, in_dim_k)
310             delta = delta @ W
311
312     return dL_dweights, dL_dbias # Moved outside the loop
313
314 def update_parameters(all_weights, all_biases, dL_dweights, dL_dbias, learning_rate):
315     """
316     Update all weights and biases using gradient descent.
317
318     Parameters:
319         all_weights: list of weight matrices (_0, _1, ..., _K)
320         all_biases: list of bias vectors (_0, _1, ..., _K)
321         dL_dweights: list of gradients w.r.t. weights (same length as all_weights)
322         dL_dbias: list of gradients w.r.t. biases (same length as all_biases)
323         learning_rate: scalar (step size)
324
325     Returns:
326         Updated all_weights and all_biases (you can modify in-place or return new lists)
327     """
328     # Number of layers
329     num_layers = len(all_weights)
330
331     # Update each layer
332     for layer in range(num_layers):
333         # Update weights: W ← W - * W
334         all_weights[layer] = all_weights[layer] - learning_rate * dL_dweights[layer]
335
336         # Update biases: b ← b - * b
337         all_biases[layer] = all_biases[layer] - learning_rate * dL_dbias[layer]
338
339     # Since we modified the lists in-place, we can just return them
340     return all_weights, all_biases
341
342 import numpy as np
343
344 def predict(X, y, all_weights, all_biases, return_probabilities=False):
345     """
346     Make predictions on a dataset and compute accuracy + average loss.
347
348     Args:
349         X: Input images, shape (N, 784)
350         y: One-hot labels, shape (N, 10)
351         all_weights: List of weight matrices

```

```

352     all_biases: List of bias vectors
353     return_probabilities: If True, also return the softmax probabilities
354
355     Returns:
356     predictions: Predicted class indices, shape (N,)
357     accuracy: Fraction of correct predictions (float)
358     cost: Average cross-entropy loss (float)
359     probabilities: (optional) softmax outputs, shape (N, 10)
360     """
361     # Forward pass on the entire dataset
362     logits, _, _ = forward_pass(X, all_weights, all_biases)
363
364     # Get predicted probabilities
365     probabilities = softmax(logits) # shape (N, 10)
366
367     # Predicted class = argmax over classes
368     predictions = np.argmax(probabilities, axis=1) # shape (N,)
369
370     # True class labels (from one-hot)
371     true_labels = np.argmax(y, axis=1) # shape (N,)
372
373     # Accuracy
374     correct = (predictions == true_labels)
375     accuracy = np.mean(correct) # or np.sum(correct) / len(X)
376
377     # Average cross-entropy loss
378     cost = compute_cost(logits, y)
379
380     if return_probabilities:
381         return predictions, accuracy, cost, probabilities
382     else:
383         return predictions, accuracy, cost
384
385 import numpy as np
386
387 def random_mini_batches(X, Y, mini_batch_size=64, shuffle=True):
388     """
389     Creates a list of random mini-batches from (X, Y)
390
391     Arguments:
392     X -- input data, shape (num_examples, input_dim) e.g. (60000, 784)
393     Y -- labels (one-hot), shape (num_examples, num_classes) e.g. (60000, 10)
394     mini_batch_size -- size of the mini-batches, integer
395     shuffle -- whether to shuffle the data before creating mini-batches (usually True)
396
397     Returns:
398     mini_batches -- list of tuples: [(x_mini1, y_mini1), (x_mini2, y_mini2), ...]
399     """
400     m = X.shape[0] # number of training examples
401     mini_batches = []
402
403     # Step 1: Shuffle the data (if shuffle is True)
404     if shuffle:
405         # Generate random permutation of indices
406         permutation = np.random.permutation(m)
407         # Shuffle both X and Y using the same permutation
408         shuffled_X = X[permutation]
409         shuffled_Y = Y[permutation]
410     else:

```

```

411         shuffled_X = X
412         shuffled_Y = Y
413
414         # Step 2: Partition into mini-batches
415         # Number of complete mini-batches
416         num_complete_batches = m // mini_batch_size
417
418         for k in range(num_complete_batches):
419             x_mini = shuffled_X[k * mini_batch_size : (k + 1) * mini_batch_size]
420             y_mini = shuffled_Y[k * mini_batch_size : (k + 1) * mini_batch_size]
421             mini_batches.append((x_mini, y_mini))
422
423         # Step 3: Handle the last (possibly smaller) mini-batch
424         if m % mini_batch_size != 0:
425             x_mini_last = shuffled_X[num_complete_batches * mini_batch_size :]
426             y_mini_last = shuffled_Y[num_complete_batches * mini_batch_size :]
427             mini_batches.append((x_mini_last, y_mini_last))
428
429         return mini_batches
430
431     # Assuming we already have:
432     # X_train, y_train, X_test, y_test = load_mnist()
433
434     batch_size = 128
435
436     # Generate mini-batches for one epoch
437     mini_batches = random_mini_batches(X_train, y_train, mini_batch_size=batch_size)
438
439     print(f"Number of mini-batches: {len(mini_batches)}")
440     print(f"First mini-batch shapes: X={mini_batches[0][0].shape}, Y={mini_batches[0][1].shape}")
441
442     # Check the last one (may be smaller)
443     print(f"Last mini-batch shapes: X={mini_batches[-1][0].shape}, Y={mini_batches[-1][1].shape}")
444
445     def train_model(
446         X_train, y_train,
447         X_test, y_test,
448         layer_sizes,                # e.g. [784, 128, 64, 10] | input + hidden + output
449         learning_rate=0.01,
450         batch_size=128,
451         num_epochs=20,
452         eval_every=100,             # evaluate every N batches
453         random_seed=42
454     ):
455         """
456         Train a fully connected neural network on MNIST.
457
458         Returns:
459             history: dict with keys:
460                 'train_losses', 'test_losses', 'train_accs', 'test_accs'
461                 (lists of values collected during training)
462         """
463         np.random.seed(random_seed)
464
465         # -----
466         # 1. Initialize model
467         # -----
468         # layer_sizes = [input_dim, hidden1, hidden2, ..., output_dim]
469         # We derive K and D from this list

```

```

470 input_dim = layer_sizes[0]          # should be 784
471 output_dim = layer_sizes[-1]        # should be 10
472 hidden_dims = layer_sizes[1:-1]    # hidden layer sizes
473
474 K = len(hidden_dims)                # number of hidden layers
475
476 # Create lists of weights and biases
477 all_weights, all_biases = [], []
478
479 prev_dim = input_dim
480 for dim in hidden_dims + [output_dim]:
481     # He initialization for weights
482     W = np.random.randn(dim, prev_dim) * np.sqrt(2.0 / prev_dim)
483     b = np.zeros((dim, 1))
484     all_weights.append(W)
485     all_biases.append(b)
486     prev_dim = dim
487
488 # -----
489 # 2. Training history
490 # -----
491 train_losses = []
492 test_losses = []
493 train_accs = []
494 test_accs = []
495
496 global_step = 0 # for eval_every
497
498 print(f"Starting training: {num_epochs} epochs, batch size {batch_size}, lr {learning_rate}")
499 print(f"Architecture: {layer_sizes}")
500
501 # -----
502 # 3. Training loop
503 # -----
504 for epoch in range(num_epochs):
505     # Shuffle and create mini-batches for this epoch
506     mini_batches = random_mini_batches(X_train, y_train, batch_size)
507
508     epoch_loss = 0.0
509     num_batches = len(mini_batches)
510
511     for i, (x_batch, y_batch) in enumerate(mini_batches):
512         global_step += 1
513
514         # Forward
515         logits, pre_acts, acts = forward_pass(x_batch, all_weights, all_biases)
516
517         # Loss
518         batch_loss = compute_cost(logits, y_batch)
519         epoch_loss += batch_loss
520
521         # Backward
522         dL_dlogits = d_cost_d_logits(logits, y_batch)
523         dweights, dbiases = backward_pass(
524             dL_dlogits, pre_acts, acts, all_weights
525         )
526
527         # Update
528         all_weights, all_biases = update_parameters(

```

```

529         all_weights, all_biases,
530         dweights, dbiases,
531         learning_rate=learning_rate
532     )
533
534     # Periodic evaluation
535     if global_step % eval_every == 0 or i == num_batches - 1:
536         # Train set performance
537         _, train_acc, train_loss = predict(X_train, y_train, all_weights, all_biases)
538
539         # Test set performance
540         _, test_acc, test_loss = predict(X_test, y_test, all_weights, all_biases)
541
542         train_losses.append(train_loss)
543         test_losses.append(test_loss)
544         train_accs.append(train_acc)
545         test_accs.append(test_acc)
546
547         print(f"[Epoch {epoch+1}/{num_epochs} | Step {global_step}] "
548               f"Train loss: {train_loss:.4f}  acc: {train_acc:.4f} | "
549               f"Test loss: {test_loss:.4f}  acc: {test_acc:.4f}")
550
551     # End-of-epoch average loss
552     avg_epoch_loss = epoch_loss / num_batches
553     print(f"Epoch {epoch+1} finished. Avg batch loss: {avg_epoch_loss:.4f}")
554
555     # Return history for plotting
556     history = {
557         'train_losses': train_losses,
558         'test_losses': test_losses,
559         'train_accs': train_accs,
560         'test_accs': test_accs
561     }
562
563     return history, all_weights, all_biases
564
565 # Example call
566 history, final_weights, final_biases = train_model(
567     X_train, y_train,
568     X_test, y_test,
569     layer_sizes=[784, 32, 10], # Small network for quick test (1 hidden layer, 32 units)
570     learning_rate=0.001,
571     batch_size=64,
572     num_epochs=25,
573     eval_every=50
574 )
575
576 # Plot results (using the provided function)
577 from utils import training_curve_plot
578
579 training_curve_plot(
580     title="MNIST Training Progress",
581     train_costs=history['train_losses'],
582     test_costs=history['test_losses'],
583     train_accuracy=history['train_accs'],
584     test_accuracy=history['test_accs'],
585     batch_size=128,
586     learning_rate=0.001,
587     num_epochs=25,

```

```

588     elapsed=0  # we can measure time if wanted
589 )
590
591 layer_sizes = [784, 10]  # input + output directly
592 history_linear, w_linear, b_linear = train_model(
593     X_train, y_train,
594     X_test, y_test,
595     layer_sizes=[784, 10],          # + linear model
596     learning_rate=0.01,             # usually higher than for deep nets
597     batch_size=128,
598     num_epochs=30,                  # 20{50 epochs is usually enough
599     eval_every=100,                 # or every epoch if preferred
600     random_seed=42
601 )
602
603 from utils import training_curve_plot
604
605 training_curve_plot(
606     title="Linear model on MNIST (no hidden layers)",
607     train_costs=history_linear['train_losses'],
608     test_costs=history_linear['test_losses'],
609     train_accuracy=history_linear['train_accs'],
610     test_accuracy=history_linear['test_accs'],
611     batch_size=128,
612     learning_rate=0.01,
613     num_epochs=30,
614     elapsed=0  # we can leave this as 0 or measure real time
615 )
616
617
618
619 import matplotlib.pyplot as plt
620
621 # w_linear is the only weight matrix (list with one element)
622 W = w_linear[0]          # shape (10, 784)
623
624 fig, axes = plt.subplots(2, 5, figsize=(12, 5))
625 axes = axes.ravel()
626
627 for i in range(10):
628     # Reshape the i-th row into 28x28 image
629     weight_image = W[i].reshape(28, 28)
630
631     # Plot
632     axes[i].imshow(weight_image, cmap='viridis') # or 'gray', 'seismic'
633     axes[i].set_title(f'Class {i}')
634     axes[i].axis('off')
635
636 plt.suptitle("Learned weight templates for each digit class (linear model)", y=1.02)
637 plt.tight_layout()
638 plt.show()
639
640 history_relu, w_relu, b_relu = train_model(
641     X_train, y_train,
642     X_test, y_test,
643     layer_sizes=[784, 128, 64, 10],
644     learning_rate=0.003,          # 0.003, 0.005, 0.01
645     batch_size=128,
646     num_epochs=50,

```



```

647     eval_every=200,
648     random_seed=42
649 )
650
651 # Plot
652 training_curve_plot(
653     title="Multi-layer NN { ReLU activation",
654     train_costs=history_relu['train_losses'],
655     test_costs=history_relu['test_losses'],
656     train_accuracy=history_relu['train_accs'],
657     test_accuracy=history_relu['test_accs'],
658     batch_size=128,
659     learning_rate=0.003,
660     num_epochs=50,
661     elapsed=0
662 )
663
664 history_sigmoid, w_sigmoid, b_sigmoid = train_model(
665     X_train, y_train,
666     X_test, y_test,
667     layer_sizes=[784, 128, 64, 10],
668     learning_rate=0.5, # much higher / 0.5, 1.0, 2.0
669     batch_size=128,
670     num_epochs=80, # often needs more epochs
671     eval_every=200,
672     random_seed=42
673 )
674
675 # Override activation to sigmoid
676
677
678 training_curve_plot(
679     title="Multi-layer NN { Sigmoid activation",
680     train_costs=history_sigmoid['train_losses'],
681     test_costs=history_sigmoid['test_losses'],
682     train_accuracy=history_sigmoid['train_accs'],
683     test_accuracy=history_sigmoid['test_accs'],
684     batch_size=128,
685     learning_rate=0.5,
686     num_epochs=80,
687     elapsed=0
688 )
689
690 history_wide_relu, w_wide, b_wide = train_model(
691     X_train, y_train,
692     X_test, y_test,
693     layer_sizes=[784, 256, 128, 64, 10], # wider & deeper
694     learning_rate=0.003, # slightly lower than before
695     batch_size=128,
696     num_epochs=60,
697     eval_every=200,
698     random_seed=42
699 )
700
701 # Plot
702 training_curve_plot(
703     title="Wider ReLU NN [784-256-128-64-10]",
704     train_costs=history_wide_relu['train_losses'],
705     test_costs=history_wide_relu['test_losses'],

```

```
706     train_accuracy=history_wide_relu['train_accs'],
707     test_accuracy=history_wide_relu['test_accs'],
708     batch_size=128,
709     learning_rate=0.003,
710     num_epochs=60,
711     elapsed=0
712 )
```
